BACKEND ENGINEERING INTERNSHIP

Building a production ERP for the *Indian jewellery trade*

A six-month engineering programme working on a real, live product — multi-branch inventory, a weight-and-money dual ledger, GST compliance, and double-entry accounting, built on PostgreSQL and TypeScript.

ROLE
Backend Developer Intern

OPENINGS
Four

DURATION
18 weeks

STACK
Node.js · TypeScript · NestJS · PostgreSQL

TEAM
4 interns + 2 senior engineers

MENTORSHIP
Daily review · weekly architecture sessions

Project brief for candidates · Issued August 2026 · Please read in full before the orientation session.

SECTION 1

About this document

This brief explains what you will be building, why it is technically interesting, how the work is divided between the four of you, and what we expect you to know before you start.

Read it properly. It is longer than a typical internship description because the product is genuinely unusual, and a large part of your first month is spent learning the *business* rather than the code. Candidates who arrive already understanding what a *tunch* or a *making charge* is will be productive weeks earlier than those who don't.

Section 11 is a preparation checklist. If you do nothing else with this document, do that.

What this is not

This is not a training exercise or a mock project. You will be writing code that a real business runs on, handling real money, and subject to Indian tax and hallmarking law. Your work is reviewed, tested and deployed. That is the point of the internship, and it is also why the standards described in Section 8 are non-negotiable.

> **A NOTE ON DIFFICULTY**
>
> Jewellery retail software is harder than it sounds, and much harder than a typical CRUD application. We know that. The work has been divided so that no intern is handed a problem they cannot finish, every module has a senior engineer attached to it, and the most dangerous business logic has already been written and tested before you arrive. Your job is to build the database, the API and the safeguards around it — which is exactly the right difficulty for someone at your stage.

SECTION 2

The product: what an Indian jewellery ERP actually does

Start here, because if you carry your instincts from ordinary e-commerce into this domain, almost everything you build will be subtly wrong.

Why a jewellery shop is not a normal shop

In ordinary retail, a product has a price. A T-shirt costs ₹499, you hold fifty of them, you sell one and now you hold forty-nine. Two facts make jewellery completely different:

One — there is no "fifty of them." Every gold ring is a unique physical object with its own weight, its own stones and its own government certificate number. Two rings made from the same design weigh different amounts and sell for different money. So there is no *quantity* column anywhere in this system. Every physical piece is one database row, which the trade calls a **tag**, after the paper tag tied to the piece in the shop.

Two — there is no fixed price. The price is calculated at the moment of sale from the day's gold rate, which the owner changes daily and sometimes twice daily. The same ring costs different money this afternoon than it did this morning.

How a piece of jewellery is priced

This formula is the heart of the product. You will see it, in one form or another, on almost every screen:

```
Net Weight    = Gross Weight − Stone Weight      stones are not gold, so they are excluded
Metal Value   = Net Weight × today's rate per gram
Wastage Value = Net Weight × wastage % × rate    gold physically lost during manufacture
Making Charge = Net Weight × ₹ per gram   OR   a flat fee
Stone Value   = the certified value of the diamonds
                ──────────────────────────────────────
Taxable Value = the sum of the above
GST @ 3%      = charged on the taxable value
Grand Total   = taxable value + GST, rounded to the rupee
```

Then the customer very often hands over an old gold chain as part of the payment. That is **not a discount** — it is the shop *buying* gold from the customer, a separate purchase transaction, deducted only at the very end after GST has already been calculated. Subtracting it before GST would understate the tax due. Small rule, large consequences.

The two parallel ledgers

This is the single most important idea in the system, and it is what makes the domain intellectually interesting.

> The business keeps two sets of books simultaneously: one in **rupees**, and one in **grams**. Both must always balance.

The weight ledger exists because gold is borrowed and repaid *as gold*. A shop owes its goldsmith four hundred grams, not ₹28 lakh — and if the gold price moves, the rupee figure changes but the debt does not. Banks lend

jewellers gold under a facility called a Gold Metal Loan, repayable in metal. So a stock item's value in rupees is always a *derived* figure, never a stored fact.

Practically, this means every money column in the database sits beside the weight it was computed from and the exact rate version used. If you store only the rupees, you cannot answer the question every shop owner asks daily — *what is my stock worth today?* — without replaying months of history.

The rest of the business you will encounter

AREA	WHAT IT INVOLVES
Tagging	Each physical piece moves through a twelve-state lifecycle: raw metal → issued to a goldsmith → received → sent for hallmarking → in stock → sold. Illegal jumps between states must be rejected, not silently allowed.
Karigar (job work)	Shops don't manufacture in-house. They issue raw gold to independent goldsmiths and receive finished pieces back, always weighing less. Reconciling that loss across mixed purities requires converting everything to a common basis called Fine Gold Equivalent.
Hallmarking	Indian law requires gold jewellery to carry a unique government ID (a HUID). It must be globally unique, can never be reused, and a non-exempt piece without one cannot legally be sold.
Old gold exchange	Customers trade in old jewellery. It is tested for purity, a melting loss is deducted, and it is bought at a rate below the selling rate. It then enters a vault and is eventually melted back into raw metal.
GST compliance	Tax splits into CGST + SGST for a sale within the state, or IGST across state lines. Invoice numbers must be strictly consecutive with no gaps, per branch, per financial year — a legal requirement, not a preference. Electronic invoices are registered with a government portal.
Accounting	Every transaction automatically posts a balanced double-entry journal. Debits must equal credits, always, with no rounding tolerance and no manual correction.
Multi-branch	A chain has several shops, each with its own tax registration and its own invoice series. Stock transfers between them are tracked in transit, and a piece must never be sellable in two places at once.
Savings schemes	Customers pay monthly instalments toward a future purchase. Legally, these must be redeemable only in jewellery and never refundable in cash — a constraint the software has to enforce structurally.

SECTION 3

Why this is a good internship, technically

Most internships hand you a to-do list application with a login screen. Here is what this project will teach you that one cannot.

YOU WILL LEARN	BECAUSE THE DOMAIN FORCES IT
Why money is never a float	`8.2 × 6650` evaluates to `54529.99999999999`. Add a thousand such figures and the error is real money. All amounts are stored as integer paisa and all weights as integer milligrams.
Database-enforced correctness	An unbalanced journal entry is prevented by a constraint in PostgreSQL, not by hoping the application is right. You will learn to make invalid states impossible rather than merely unlikely.
Real concurrency	Two billing counters must never sell the same piece, and must never receive the same invoice number. This is row locking, transactions and isolation levels applied to a problem where getting it wrong is visible to a customer.
Multi-tenant data isolation	Many businesses share one database. You will use PostgreSQL Row-Level Security so that isolation is enforced by the database itself, rather than depending on every developer remembering a filter.
Append-only and audit design	Rates, ledgers and audit trails are never updated or deleted, only added to. You will learn why regulated systems are built this way and how to design around it.
Asynchronous integration	Government tax APIs are slow and periodically down. The billing counter must never wait for them. You will build a transactional outbox and a background job queue.
Testing that matters	A silent bug in accounting software compounds daily instead of failing loudly once. You will write tests because you want them, not because a checklist asked.
Working to a specification	Tax law, hallmarking rules and GST invoice format are external requirements you cannot negotiate with. This is the closest most engineers get to genuine specification-driven development.

> **WHAT ALREADY EXISTS**
>
> A working front-end application and a substantial, well-tested library of business rules — the pricing engine, the purity mathematics, the tax splitting, the double-entry posting — already exist and are covered by an extensive automated test suite. You will not be asked to invent Indian jewellery accounting from scratch.
>
> Your work is to give that logic a database, a secure multi-tenant API and the safeguards a production system needs. This is deliberate: it removes the part of the problem that would be unreasonable to hand to an intern, and leaves the part that will genuinely teach you backend engineering.

SECTION 4

Technology stack

Learn the reasons, not just the names. You will be asked why we chose each of these, and "it's popular" is not an answer we accept.

TECHNOLOGY	ROLE	WHY THIS ONE
Node.js 22 LTS	Runtime	The existing business logic is already TypeScript. Rewriting a large, tested rules engine in another language would discard the most valuable asset in the project to gain nothing.
TypeScript 5.8	Language	Strict mode throughout. In a domain where a wrong number is a wrong invoice, a compiler that catches mistakes is worth the ceremony.
NestJS	Web framework	Opinionated by design. It tells you where controllers, services and dependency injection go, which is exactly what a team of four people building in parallel needs. You will use its modules, guards, interceptors and pipes.
PostgreSQL 16	Database	Chosen over MongoDB for four specific reasons: journals must balance (enforceable with a constraint), money must not drift (exact integer types), invoice numbers must be gap-free (row locking), and every report is relational aggregation.
Drizzle ORM	Query layer	Its queries read like SQL, so a reviewer can see exactly what hits the database. Migrations are reviewable SQL rather than opaque generated artefacts.
Redis	Cache	The gold rate is read on every billing line. It is served from cache in single-digit milliseconds and never from the database on the hot path.
BullMQ	Job queue	Government API calls, messaging and exports run in a background worker so the billing counter never blocks on an external system.
Zod	Validation	One schema definition validates the API request and types the front-end client, so the two cannot drift apart.
Vitest + Testcontainers	Testing	Fast unit tests with no database, plus integration tests against a real PostgreSQL instance started automatically for each run.

TECHNOLOGY	ROLE	WHY THIS ONE
Docker	Local environment	One command gives you PostgreSQL and Redis locally. No "works on my machine."
GitHub Actions	CI	Every pull request runs lint, type-check, unit tests, integration tests, migration validation and security scanning before a human looks at it.

> **IF YOU COME FROM THE MERN STACK**
>
> Most of what you know transfers directly. Express middleware becomes NestJS guards and interceptors. Mongoose models split into a database schema, a validation schema and a set of pure business functions. Populate becomes a join. The concepts are the same; the guarantees are stronger. Section 11 tells you exactly what to read before you start.

SECTION 5

How the code is organised

One rule governs the entire codebase. Learn it now and most code review comments you receive will make immediate sense.

The layer contract

Every feature is built from the same five layers, in the same order, with the same file names. Two further layers are shared across the whole system and provided to you.

1 · Controller `tags.controller.ts`	Handles HTTP only. Reads the request, calls exactly one service method, returns the result. It must not contain a decision about business meaning.
2 · DTO `dto/create-tag.dto.ts`	Declares the shape of the request and response and validates it at the boundary. No logic.
3 · Service `tags.service.ts`	Orchestrates the use case. Opens the database transaction, calls repositories, calls business-rule functions, publishes events. Contains *no arithmetic*.
Business rules `shared package`	Pure functions — data in, data out, no database, no framework. Every price, tax split, purity conversion and posting rule lives here. You import these; you never rewrite them.
4 · Repository `tags.repository.ts`	All database access. Every query is scoped to the current business and branch. It never decides what *should* happen.
5 · Schema `db/schema/tags.ts`	Table definitions — columns, types, constraints, indexes. Nothing else. No hooks, no methods.
Platform `shared package`	Authentication, tenant context, audit logging, error handling, idempotency. Wraps every request automatically.

> Services have no arithmetic. Repositories have no decisions.

Those two sentences resolve almost every "where does this code go?" question you will have in your first month.

Folders follow features, not layers

If you have built a Node backend before, you have probably used folders named `controllers/`, `services/` and `models/`, each containing every file of that type. That works well up to roughly a dozen features. This system has around ninety database tables, so we organise the other way round — everything about tags lives in one folder:

```
src/
├── tags/
│   ├── tags.controller.ts
│   ├── tags.service.ts
│   ├── tags.repository.ts
│   └── dto/
├── transfers/
│   └── … same five files …
├── db/
│   ├── schema/
│   └── migrations/
└── config/
```

One folder to open. One folder that appears in your diff. Four people working simultaneously without stepping on each other.

Naming conventions, fixed

ITEM	CONVENTION
Database table	`snake_case`, plural — `tags`, `sale_invoices`
Database column	`snake_case` with a unit suffix — `net_weight_mg`, `metal_value_paisa`
API endpoint	`/v1/` plus a plural kebab-case noun — `/v1/item-designs`
Error code	`SCREAMING_SNAKE_CASE` — `TAG_ILLEGAL_TRANSITION`
Branch name	`feat/inventory-tag-transitions`
Commit message	Conventional Commits — `feat(tags): reject sale of in-transit piece`

The unit suffix on columns is not optional. A column called `weight` or `amount` will be sent back in review; `net_weight_mg` tells every future reader the unit without opening a document.

SECTION 6

The four tracks

The system is divided into four independent areas. Each of you will own one, end to end — the database tables, the business logic wiring, the API endpoints and the tests. Assignment happens on your first day, after we have talked to you about your interests and strengths.

You own your track. That means you are the person who understands it best, who explains it in review, and whose name is on it. It also means the other three depend on you delivering an agreed interface on time.

TRACK A Identity, Access & Master Data

What it covers. Who is allowed to do what, and the reference data every other part of the system depends on: the metals and purities the business deals in, the daily gold rate, tax rates and HSN codes, branches, and the customer/supplier register.

WHAT YOU WILL BUILD

- Operator accounts, roles and a permission system with a branch dimension — a manager in one city must not act in another
- An approval workflow where a large discount needs a second person's sign-off, and self-approval is refused
- An append-only gold rate history, so an invoice from six months ago still resolves the rate it was billed at
- Effective-dated tax rates, because GST rates change by government notification
- A tamper-evident audit trail of every sensitive action
- Later: the customer savings-scheme module

WHAT MAKES IT HARD

- Everyone else is blocked until your data model is right, so this track starts first and carries real responsibility
- Append-only design: you can add a rate but never edit one, enforced by the database itself
- Caching the rate correctly — it is read on every billing line and must never be stale
- No legal threshold may ever be hard-coded; all of them are configurable data

SKILLS YOU WILL DEVELOP

Authentication & JWT | Role-based access control | Temporal / effective-dated data | Cache invalidation | Audit design

TRACK B

Inventory, Tagging & Production

What it covers. The physical goods. Every individual piece of jewellery in the business, its complete lifecycle, movements between branches, stock audits, and the relationship with the goldsmiths who manufacture the pieces.

WHAT YOU WILL BUILD

- The tag — one row per physical piece, with weights stored in milligrams
- A twelve-state lifecycle where every illegal transition is rejected with a clear error
- Barcode and QR label generation
- Inter-branch transfers with an in-transit state, and per-piece acceptance at the destination
- Physical stock audit with discrepancy reporting
- Goldsmith job work: issuing raw gold, receiving finished pieces, reconciling the weight lost in manufacture
- Hallmarking dispatch and unique government ID assignment

WHAT MAKES IT HARD

- Concurrency. Two counters must never sell the same piece — this is real row locking, and it is tested with fifty simultaneous requests
- A piece may be sellable at exactly one branch, enforced by a database index rather than by convention
- Stock is not uniformly owned — some is financed, some belongs to a supplier until sold, and the difference changes the balance sheet
- Weight reconciliation across mixed purities requires converting everything to a common basis

SKILLS YOU WILL DEVELOP

State machines | Row locking & transactions | Optimistic concurrency | Partial indexes | Event-driven design

TRACK C

Billing, Point of Sale & Old Gold

What it covers. Every rupee the business takes. Quotations, tax invoices, discounts and price overrides, split payments, returns and credit notes, and the old-gold exchange that so many sales involve.

WHAT YOU WILL BUILD

- The invoice endpoint — calculating every line through the shared pricing engine
- Quotation mode with its own non-tax numbering series, convertible into a real invoice
- Split payment across cash, card, UPI and scheme redemption, validated to settle exactly
- Price override with a mandatory logged reason and, above a limit, a supervisor's approval
- Statutory gates: identity documents required above a cash threshold; an uncertified piece cannot be billed
- Sales returns and credit notes with proportional discount reversal
- Old gold: purity testing, melting-loss deduction, valuation, and a vault lifecycle
- An offline synchronisation endpoint for counters that lose connectivity

WHAT MAKES IT HARD

- Invoice numbers must be strictly consecutive with no gaps, per branch, per year — under concurrent load, by law
- Every write must be safely repeatable, because an offline counter retries and a retried sale must not become two invoices
- The old-gold trade-in must never touch the taxable value, only the final amount collected
- A finalised invoice can never be edited — corrections are separate documents
- A calculation must complete in under 200 milliseconds so checkout stays fast

SKILLS YOU WILL DEVELOP

Idempotency | Sequence generation under load | Transaction boundaries | Performance work | Offline sync & conflict resolution

TRACK D

Procurement, Tax & Accounting

What it covers. Buying, tax compliance and the books. Purchase orders and goods receipts, GST registers and returns, electronic invoicing with the government portal, and the double-entry ledger behind everything.

WHAT YOU WILL BUILD

- Purchase orders, goods receipts and supplier invoices, with input tax credit tracked separately for credit claimed and credit retained
- GST registers and the data behind statutory monthly returns
- Electronic invoice registration with the government portal — asynchronously, so the billing counter never waits on it
- The chart of accounts and automatic journal posting behind every transaction
- Trial balance, ledger statements, receivables ageing, profit & loss and balance sheet
- Export to Tally, which most Indian accountants still use
- Accounting period locking

WHAT MAKES IT HARD

- Debits must equal credits, always. This is enforced in three independent places so that a bug simply cannot commit an unbalanced entry
- Routine transactions may never create a journal entry without a source document behind it
- A posted entry is never edited or deleted — a correction is a reversal entry
- Stock financed by a metal loan appears as a liability measured in *grams*, not rupees
- Government APIs are slow and sometimes offline; the system must degrade gracefully and reconcile later

SKILLS YOU WILL DEVELOP

Double-entry accounting · Database constraints & triggers · Materialised views · Transactional outbox · Third-party API integration

TRACK D AND MENTORSHIP

The accounting track carries the highest correctness risk in the system, so it receives the most senior support: a full week of pair programming at the start of the accounting phase, and individual review of every posting change by a senior engineer. Nobody is left alone with the balance sheet.

SECTION 7

Timeline

Eighteen weeks. The first two are deliberately not feature work — you will be learning the domain and the stack while the senior engineers build the shared foundation everything else sits on.

WEEKS	PHASE	WHAT HAPPENS
0	**Orientation**	Domain training, environment setup, conventions, tooling. You will compute a full jewellery invoice by hand before writing any code.
1 – 2	**Foundation**	Seniors build the shared platform, security layer and a complete reference module. You study that reference module — it is the pattern every track copies. Training in NestJS, Drizzle and PostgreSQL security.
3 – 4	**Interface design**	Each of you designs the API contract for your track, with a senior engineer. This is agreed and merged *before* implementation, so the four tracks can then proceed in parallel without waiting for each other.
3 – 9	**Foundations of each track**	Track A builds identity and master data. Track B builds tags and lifecycle. Tracks C and D begin as their dependencies land.
6 – 14	**Core delivery**	All four tracks running in parallel. Billing, procurement, production and tax. Regular integration and the first end-to-end business flows.
13 – 18	**Financial close**	Accounting, old gold, schemes and reporting. The system starts behaving as one product rather than four.
16 – 18	**Hardening**	Load testing at ten times normal volume, security review, disaster-recovery rehearsal, documentation, release.

The daily and weekly rhythm

WHEN	WHAT
Daily, 09:45	Fifteen-minute stand-up, standing. Three questions only: what moved, what is blocked, whose pull request needs review. Blockers are assigned an owner in the meeting, not debated in it.
Daily, 10:00–12:00	Senior engineers review code. You get a first response on any pull request within four working hours — being blocked on review is not something you should ever have to chase.
Friday afternoon	Ninety minutes together: architecture decisions, a review retrospective, and a walkthrough of every database change made that week. The database session is taught in front of everyone, so you learn from the other three tracks as well as your own.

SECTION 8

How we work

Git and pull requests

- You work on short-lived branches off `main`. A branch older than three days is raised at stand-up — that is a review problem, not a coding problem.
- Nobody pushes directly to `main`. Every change goes through a pull request with automated checks and at least one senior approval.
- Changes touching money, security or the database schema require two approvals.
- Commit messages follow Conventional Commits and explain *why*, not just what.

What runs automatically on every pull request

Lint · formatting · type-check · unit tests · integration tests against a real database · data-isolation test · API contract validation · database migration validation · dependency vulnerability scan · static security analysis · secret scanning. The whole pipeline finishes in under ten minutes. If it fails, a human does not look at your change until it passes.

Definition of done

A feature is not finished because the endpoint returns a 200. It is finished when all of the following are true:

- ☐ Unit tests for every service method, including the failure paths
- ☐ Integration tests against a real database for every endpoint
- ☐ Every database query correctly scoped to the right business and branch
- ☐ All input validated at the boundary
- ☐ Error responses use defined codes with messages a shopkeeper could act on
- ☐ Loading, empty, forbidden, conflict and not-found paths all handled — no silent failures
- ☐ Every state change written to the audit trail with an actor and a reason
- ☐ Permission checked on the server, never only in the interface
- ☐ Database migration reviewed as SQL, with justified indexes
- ☐ No floating-point arithmetic anywhere near money or weight
- ☐ No business rule duplicated that already exists in the shared engine
- ☐ API documentation regenerated and the module's README updated
- ☐ You can explain the module out loud to a senior engineer, including the business rule behind it

That last one matters more than it looks. If you cannot explain a rule in plain English, you have not understood it — and finding that out in a handover conversation is far cheaper than finding it out in production.

> **ON HAVING YOUR WORK SENT BACK**
>
> Your pull requests will be rejected. Regularly, in your first month, all four of you. A change bounced for a missing data scope or a stray calculation is the process working exactly as intended — it is not a judgement of you, and it is not something to be embarrassed about. Reviews here explain the reasoning and point you at the right pattern, because with four interns on a domain this unforgiving, code review *is* the training programme.

The mistakes that come back most often

INSTEAD OF	DO
Writing your own price calculation	Call the shared pricing engine — it already exists and is tested
Hard-coding a legal threshold or tax rate	Read it from configuration; those numbers change by government notification
Storing money as a decimal or float	Integer paisa. Weights in integer milligrams.
Comparing two computed amounts with `===`	Use the provided comparison helper — floating point makes equality unreliable
Updating a rate row	Insert a new version. History is never rewritten.
Deleting a record	Mark it inactive with a reason. Deleting a written-off piece deletes the record of the loss.
Awaiting a government API inside a request	Queue it. The counter never waits for an external system.
Business logic in a controller	Push it down to the service, or further down to the shared rules engine

SECTION 9

What we expect from you

Required before you start

AREA	LEVEL EXPECTED
JavaScript	Comfortable. Promises, `async`/`await`, modules, array methods, destructuring. You should be able to explain what `await` actually does.
Node.js & a web framework	You have built at least one REST API — Express, NestJS or similar. You know what middleware is and what a route handler does.
Databases	Basic SQL: `SELECT`, `JOIN`, `WHERE`, `GROUP BY`. If your experience is MongoDB only, that is acceptable, but see the preparation checklist.
Git	Branch, commit, push, pull request, resolve a merge conflict without panic.
Reading code	Willing to open an unfamiliar file and work out what it does, rather than waiting to be told.

Helpful but not required

TypeScript · PostgreSQL specifically · NestJS · Docker · writing automated tests · any accounting knowledge. We will teach all of these. Arriving with any of them means you get productive faster.

What matters more than any of the above

- **Care about correctness.** This software handles other people's money. Someone who checks their work twice is worth more here than someone who writes twice as fast.
- **Ask early.** Two hours stuck is normal engineering. Two days stuck in silence is not, and it is never held against you to ask.
- **Read the business rule before writing the code.** Almost every serious defect in this domain comes from implementing a misunderstood requirement perfectly.
- **Take review well.** You will get a lot of it. That is the value of the internship.
- **Write things down.** If you worked something out the hard way, put it in the README so the next person does not.

SECTION 10

How you will be assessed

Assessment is continuous, through the work itself. There is no separate examination.

DIMENSION	WHAT WE LOOK AT
Correctness	Does the code implement the business rule as specified? Are the edge cases handled? Do the tests assert the *rule*, or only that the code runs?
Independence over time	We expect heavy support in weeks 1–4 and much less by week 12. The trajectory matters far more than the starting point.
Response to review	Does the same comment need making three times, or once? Do you understand the reasoning or just apply the fix?
Communication	Can you explain your module out loud? Do you flag a slip early or hope it resolves itself?
Reliability	Does agreed work land when you said it would? The other three tracks depend on your interfaces.
Ownership	Do you fix the underlying cause, or only the reported symptom? Do you leave your module better documented than you found it?

What you take away

- Eighteen weeks of commit history on a real production system, which you can talk about in any interview for the rest of your career
- Genuine PostgreSQL depth — transactions, locking, constraints, row-level security, indexing — which is rare in graduates and valued everywhere
- Experience of a regulated financial domain: audit trails, append-only data, compliance as a design constraint
- Habits that transfer: contract-first design, meaningful tests, reviewable migrations, honest estimates
- A written reference and a completion report describing specifically what you built

SECTION 11

Before you start — preparation checklist

If you do one thing with this document, do this. Everything below is free, and none of it takes more than a few evenings. Arriving with this done will put you weeks ahead.

TECHNICAL — IN PRIORITY ORDER

- ☐ **TypeScript basics.** Types, interfaces, generics, `strict` mode. The official handbook's first five chapters are enough.
- ☐ **SQL fundamentals.** If you have only used MongoDB, this is your single highest-value preparation. `JOIN`, `GROUP BY`, foreign keys, indexes, and what a transaction is.
- ☐ **Install PostgreSQL locally and use it.** Create tables, insert rows, write joins. Break things. An afternoon is enough.
- ☐ **The NestJS "first steps" tutorial.** Just enough to recognise controllers, providers, modules and dependency injection.
- ☐ **Install Docker Desktop** and run a PostgreSQL container. You will do this on day one; do it now so a day-one problem is not a day-one blocker.
- ☐ **Write a test.** If you have never written an automated test, write five for any function you have already built. Any framework.
- ☐ **Read about floating-point arithmetic.** Understand why `0.1 + 0.2 !== 0.3`. It comes up in your first week.

DOMAIN — DO NOT SKIP THIS

- ☐ **Learn the vocabulary.** Gross weight, net weight, stone weight, purity, 916 and 22-karat, making charge, wastage, tunch, HUID, karigar. An hour of reading.
- ☐ **Understand GST at a basic level.** What CGST, SGST and IGST are and when each applies. One article is enough.
- ☐ **Learn what double-entry bookkeeping is.** Debit, credit, why they must balance, what a ledger and a trial balance are. This is the concept most computer-science graduates have never met, and it underpins a quarter of the system.
- ☐ **Visit a jewellery shop** and look at an actual bill. Notice the weight columns, the making charge line and the hallmark. Fifteen minutes, and it will make Section 2 real in a way no document can.

PRACTICAL

- ☐ A GitHub account you will keep using
- ☐ A working development machine with at least 8 GB of RAM — Docker, PostgreSQL and your editor run simultaneously
- ☐ Node.js 22 LTS installed
- ☐ Visual Studio Code, or an editor you are genuinely fast in

ONE EXERCISE WORTH DOING

Take the pricing formula from Section 2 and implement it as a small TypeScript function, with tests. Use a 24-gram net weight, a rate of ₹6,200 per gram, 5% wastage, ₹550 per gram making charge and ₹1,20,000 of diamonds, at 3% GST. Work out what the correct total should be by hand first, then make your code agree with it.

It is thirty lines of code. Doing it before you arrive means the first real thing you read in the codebase will already be familiar — and it is the exact calculation we will walk through together on day one.

Questions before you start? Send them through your college placement office and they will reach the engineering team. Nothing in this document is a trick question, and there is no such thing as an obvious one at this stage.

We are looking forward to working with you.